

5장 합성곱 신경망 2

① 생성일 @2025년 3월 31일 오후 2:27

3 전이 학습

1. 특성 추출 기법
2. 미세 조정 기법

4 설명 가능한 CNN

1. 특성 맵 시각화

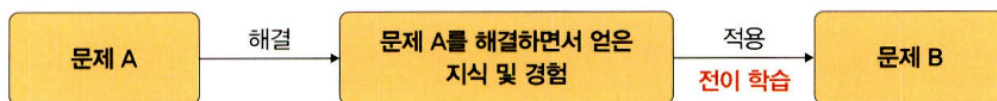
5 그래프 합성곱 네트워크

1. 그래프란
2. 그래프 신경망
3. 그래프 합성곱 네트워크

3 전이 학습

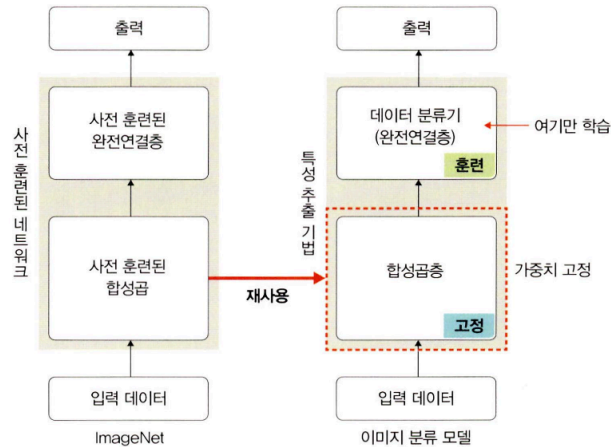
합성곱 신경망 기반 딥러닝 모델 훈련 - 많은 양의 데이터 필요 → but, 데이터 셋 확보 어려움

⇒ 전이 학습으로 이를 해결



- 전이 학습 : 아주 큰 데이터셋을 사용해 훈련된 모델의 가중치를 가져와 해결하려는 과제에 맞게 보정하여 사용하는 것을 의미 → 적은 수의 데이터로도 과제 수행 가능
- 아주 큰 데이터셋을 사용해 훈련된 모델 = 사전 훈련된 모델(네트워크)
- 전이 학습 방법 : 특성 추출 기법 / 미세 조정 기법

1. 특성 추출 기법



: ImageNet 데이터셋으로 사전 훈련된 모델을 가져옴 → 마지막에 완전 연결층 부분만 새로 만듦

⇒ 즉, 학습할 때는 마지막 완전연결층(이미지의 카테고리를 결정하는 부분)만 학습하고 나머지 계층은 학습 X

- 특성 추출 구성
 - 합성곱층: 합성곱층+풀링층
 - 완전연결층 (=데이터분류기): 추출된 특성으로 최종적으로 이미지에 대한 클래스 분류
- 사전 훈련된 네트워크의 합성곱층(가중치고정)에 새로운 데이터를 통과 → 그 출력을 데이터 분류기에서 훈련시킴
- 사용가능한 이미지 분류모델: Xception, Inception V3, ResNet50, VGG16, VGG19, MobileNet

코드 5-12 라이브러리 호출

코드 5-13 이미지 데이터 전처리 방법정의

1 torchvision.transform

```
transform = transforms.Compose([transforms.Resize([256, 256]),  
                                (a)  
                                transforms.RandomResizedCrop(224), transforms.RandomHorizontalFlip(),  
                                (b) (c)  
                                transforms.ToTensor()])  
                                (d)
```

1. **torchvision.transform** : 이미지 데이터 변환하여 모델(네트워크)의 입력으로 사용할 수 있게 변환
 - a. **Resize** : 이미지 크기 조정
→ 합성곱층 통과를 위해 이미지 크기를 조정하는 전처리 과정
 - b. **RandomResizedcrop** : 이미지 랜덤한 크기 및 비율로 자름
→ 데이터 확장 용도, 이미지 자른 후 데이터 크기 조정
⇒ **Resize** , **RandomResizedcrop** 모두 이미지 자르는 데 사용 but, 용도가 다름
 - c. **RandomtorizontalF1ip** : 이미지를 랜덤하게 수평으로 뒤집음
 - d. **ToTensor** : 이미지 데이터 텐서로 변환

2 datasets.ImageFolder

```
train_dataset = torchvision.datasets.ImageFolder(data_path,  
                                                  (a)  
                                                  transform=transform)  
                                                  (b)
```

2. **datasets.ImageFolder** : 데이터 로더가 데이터를 불러올 대상(or경로)과 방법(or 전처리)을 정의함
 - a. 첫번째 파라미터 : 불러올 데이터 위치 경로
 - b. **transform** : 이미지 데이터에 대한 전처리

3 데이터로더 (train_loader)

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32,  
                                           ①          ②  
                                           num_workers=8, shuffle=True)  
                                           ③          ④
```

3. 데이터 로더는 데이터를 불러오는 부분, 앞에서 정의한 `datasets.ImageFolder` (`train_dataset`)을 데이터로더에 할당

→ 한 번에 불러올 데이터양을 결정하는 `batch-size` 지정, 데이터를 무작위로 섞을 것인지 설정

a. 첫번째 파라미터: 데이터셋 지정

b. `batch_size` : 한 번에 불러올 데이터양 결정 = 배치 크기 설정

c. `num_workers` : 데이터 불러올 때 하위프로세스 몇 개 사용할지 설정

→ 너무 많은 하위 프로세스 설정 시 오류 발생 or 메모리 부족 현상 발생 가능

d. `shuffle` : 데이터를 무작위로 섞을지 여부 설정

= True : 데이터 무작위로 섞어 랜덤으로 불러옴



RandomResizedCrop 자세히 알아보기

```
import matplotlib.pyplot as plt
import mnet as mx
from mxnet.gluon.data.vision import transforms

example_image = mx.image.imread("../chap05/data/cat.jpg")
plt.imshow(example_image.asnumpy()) # 불러온 이미지 출력

# 불러온 이미지에 RandomResizedCrop을 적용하기 위한 함수 생성

def show_images(imgs, num_rows, num_cols, scale=2):
    aspect_ratio = imgs[0].shape[0]/imgs[0].shape[1] # 확장할 이미지
    figsize = (num_cols * scale, num_rows * scale * aspect_ratio)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    for i in range(num_rows):
        for j in range(num_cols):
            axes[i][j].imshow(imgs[i * num_cols + j].asnumpy())
            axes[i][j].axes.get_xaxis().set_visible(False) # x축 전체 숨
            axes[i][j].axes.get_yaxis().set_visible(False) # y축 전체 숨
    plt.subplots_adjust(hspace=0.1, wspace=0)
    return axes

def apply(img, aug, num_rows=2, num_cols=4, scale=3):
    Y = [aug(img) for _ in range(num_rows * num_cols)] # 다양한 샘플 얻
    show_images(Y, num_rows, num_cols, scale)

# RandomResizedCrop을 적용된 이미지 출력

shape_aug = transforms.RandomResizedCrop(size=(200, 200), # 출력
                                           scale=(0.1, 1), # 면적
                                           ratio=(0.5, 2)) # 면적

apply(example_image, shape_aug)
```

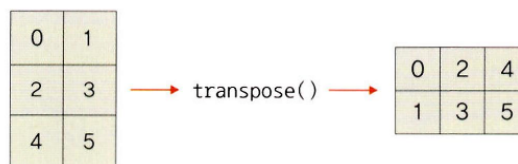
코드 5-14 학습에 사용될 이미지 출력

```
samples, labels = next(iter(train_loader))
# 최신 PyTorch 버전에서는 .next() 대신 next(iter(train_loader))를 사용
```

- 반복자(iterator, for 구문과같은효과)를 사용하려면 `iter()`, `next()` 필요
 - `iter()` : 전달된 데이터의 반복자를 꺼내 반환
 - `next()` : 그 반복자가 다음에 출력해야 할 요소 반환
- `iter()` 로 반복자 구하고 그 반복자를 `next()` 에 전달하여 차례대로 꺼낼 수 있음
- 위 코드 기준, 반복자는 `train_loader`가 되기 때문에 `train_loader`에서 `samples`와 `labels`의 값을 순차적으로 꺼내서 저장
 - ⇒ 간단 정리: `train_loader`에서 데이터를 하나씩 꺼내오겠다는 의미

```
a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))
```

- `np.transpose` : 행과 열을 바꿈으로써 행렬의 차원을 바꾸어줌



- 행렬의 차원을 변경하는 이유 : **행렬의 내적** 연산 때문

곱셈, 덧셈

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

A B C

$$\begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}$$

- A라는 행렬과 B라는 행렬의 내적 결과는C
- 이때 A행렬의 행과 B행렬의 열의 수가 같아야 내적 가능
- A와 B의 행과 열의 수가 달라 내적 불가할 시, `np.transpose()` 나 `np.reshape()` 등으로 차원 조정

- `print(samples.shape)`을 통해 데이터의 형태를 확인 → `torch.Size([32, 3, 24, 224])`가 출력
→ `np.transpose`를 적용X ⇒ *Invalids hape(3, 224, 224)for image data* 와 같은 오류가 발생
∴ `np.transpose (samples[1].numpy(), (1,2,0))` 을 사용해서(224, 224, 3)과 같은형태로 변환한 후 사용

코드 5-15 사전 훈련된 모델 내려받기



ResNet18

- ResNet18은 50개의 계층으로 구성된 합성곱신경망
- ImageNet 데이터베이스의 100만 개가 넘는 영상을 이용하여 훈련된 신경망, 전이학습에 사용되도록 사전 훈련된 모델을 제공하고 있음
- But, ResNet18은 입력 제약이 매우 크고, 충분한 메모리(RAM)가 없으면 학습 속도가 느릴 수 있는 단점이 있음



사전 훈련된 모델

- 파이토치에서 무작위 가중치 모델 구성 방법

```
import torchvision.models as models

resnet18 = models.resnet18()
alexnet = models.alexnet()
vgg16 = models.vgg16()
squeezenet = models.squeezenet1_0()
densenet = models.densenet161()
inception = models.inception_v3()
googlenet = models.googlenet()
shufflenet = models.shufflenet_v2_x1_0()
mobilenet_v2 = models.mobilenet_v2()
mobilenet_v3_large = models.mobilenet_v3_large()
mobilenet_v3_small = models.mobilenet_v3_small()
resnext50_32x4d = models.resnext50_32x4d()
wide_resnet50_2 = models.wide_resnet50_2()
mnasnet = models.mnasnet1_0()
```

- 사전 학습된 모델(사전 학습된 모델의 가중치값) 사용 방법 ⇒ pretrained=True 설정

```
import torchvision.models as models

resnet18 = models.resnet18(pretrained=True)
alexnet = models.alexnet(pretrained=True)
vgg16 = models.vgg16(pretrained=True)
squeezenet = models.squeezenet1_0(pretrained=True)
densenet = models.densenet161(pretrained=True)
inception = models.inception_v3(pretrained=True)
googlenet = models.googlenet(pretrained=True)
shufflenet = models.shufflenet_v2_x1_0(pretrained=True)
mobilenet_v2 = models.mobilenet_v2(pretrained=True)
mobilenet_v3_large = models.mobilenet_v3_large(pretrained=True)
mobilenet_v3_small = models.mobilenet_v3_small(pretrained=True)
resnext50_32x4d = models.resnext50_32x4d(pretrained=True)
```



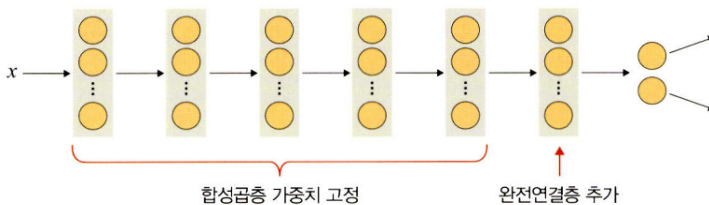
```
wide_resnet50_2 = models.wide_resnet50_2(pretrained=True)
mnasnet = models.mnasnet1_0(pretrained=True)
```

코드 5-16 사전 훈련된 모델의 파라미터 유무 지정

```
param.requires_grad = False
```

- 역전파 중 파라미터들에 대한 변화 계산 필요 X 의미
= 모델의 일부를 고정하고 나머지를 학습하고자 할 때 `requires_grad=False` 설정
→ 모델의 일부: 합성곱층 / 풀링층

코드 5-17 ResNet18에 완전연결층 추가



코드 5-18 모델의 파라미터 값 확인

코드 5-19 모델 객체 생성 및 손실함수 정의

코드 5-20 모델 학습을 위한 함수 생성

코드 5-21 파라미터 학습 결과를 옵티마이저에 전달

- 완전연결층의 어떤 파라미터들이 옵티마이저로 전달 되는지를 보여줌
→ weight 와 bias 값들이 업데이트 되고 옵티마이저에 전달될 것

코드 5-22 모델 학습

- 모델 훈련을 위해 전달되는 파라미터: 모델, 학습데이터, 손실함수, 옵티마이저, 장치(CPU or GPU)

→ 정확도 94%

코드 5-23 테스트 데이터 호출 및 전처리

코드 5-24 테스트 데이터 평가 함수 생성

```
saved_models = glob.glob('catanddog/' + '*.pth')
```

- `glob` : 현재 디렉터리에서 원하는 파일들만 추출하여 가져올 때 사용
→ 즉, 해당 경로에서 pth 확장자를 갖는 파일을 가져오라는 의미

- pth 확장자를 갖는 파일: 훈련 데이터로 모델을 훈련시킬 때 생성된 파일

```
_, preds = torch.max(outputs.data, 1)
```

- `torch.max` : 주어진 텐서 배열의 최댓값이 들어있는 index를 반환하는 함수
- ex. `y_pred = [ [ 0.2 , 0.7 , 0.8 , 0.4 ] ]` 라는 배열에 `torch.max(y_pred.data, 1)` 적용 → 해당 배열의 최댓값은 0.8 , 그 인덱스는 2 → 최종 반환값은 2

```
running_corrects += preds.eq(labels.cpu()).int().sum()
```

- `preds.eq(labels)` : preds 배열과 labels가 일치하는지 검사하는 용도
- `sum()` : 모델의 예측결과와 정답(레이블)이 일치하는 것들의 개수 합을 숫자로 출력

코드 5-25 테스트 데이터를 평가함수에 적용

코드 5-26 훈련과 테스트 데이터의 정확도를 그래프로 확인

- 훈련과 테스트 데이터 모두 에포크가 진행될수록 정확도가 높아지면서 100%에 가까워짐
→ 실제로 더 많은 에포크를 진행한다면 더욱더 100%에 수렴할 것

코드 5-27 훈련 데이터의 오차에 대한 그래프 확인

코드 5-28 예측 이미지 출력을 위한 전처리 함수

```
image = tensor.clone().detach().numpy()
```

- `tensor.clone()` : 기존 텐서의 내용을 복사한 텐서를 생성하겠다는 의미
- `tensor.detach()` : 기존 텐서에서 기울기가 전파되지 않는 텐서

⇒ 즉, `tensor.clone().detach()` 는 기존 텐서를 복사한 새로운 텐서를 생성하지만 기울기에 영향을 주지는 않겠다는 의미

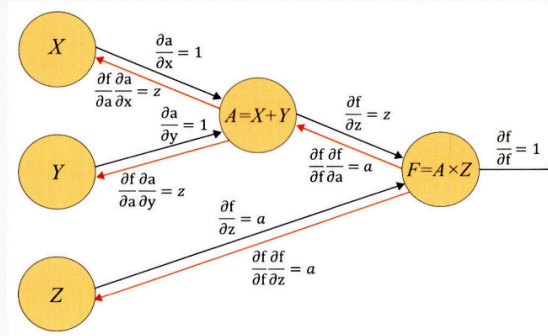
- `tensor.clone()` , `tensor.detach()` , `tensor.clone().detach()` 비교

구분	메모리	계산 그래프 상주 유무
<code>tensor.clone()</code>	새롭게 할당	계산 그래프에 계속 상주
<code>tensor.detach()</code>	공유해서 사용	계산 그래프에 상주하지 않음
<code>tensor.clone().detach()</code>	새롭게 할당	계산 그래프에 상주하지 않음



계산 그래프

: 계산 과정을 그래프로 나타낸 것, 여러 개의 노드(node)와 그 노드들을 연결하는 선인 에지(edge)로 구성



- 계산 그래프 사용 이유
 - 국소적 계산이 가능
 - Z 값이 변경 되었다면 X, Y 계산결과를 그대로 유지한 채로 바뀐 Z의 연산이 필요한 $F=4 \times 2$ 만 계산하면 됨
 - 역전파를 통한 미분 계산이 편리
 - 주황색 선이 역전파를 구하는 과정: 연쇄법칙(chain rule)을 이용하여 빠르고 간편하게 미분 계산 가능



연쇄 법칙

: 합성 함수 (= 두 개 이상의 함수가 결합된 함수) 의 미분법

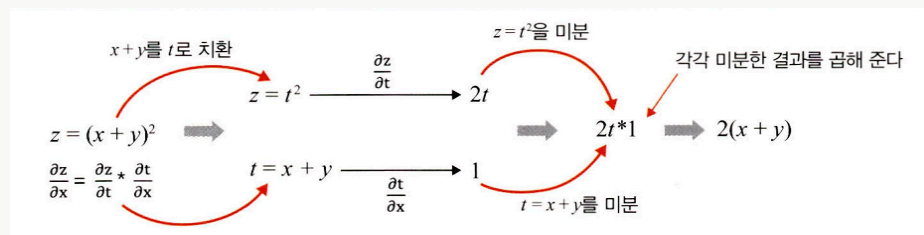


image = image.clip(0, 1)

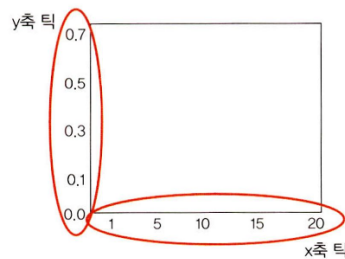
- `clip()` : 입력값이 주어진 범위 벗어날 때 입력 값을 특정 범위로 제한시키기 위해 사용

→ `image.clip(0, 1)` 은 데이터를 0과 1 사이의 값으로 제한하겠다는 의미

코드 5-29 개와 고양이 예측결과 출력

```
ax = fig.add_subplot(2, 10, idx+1, xticks=[], yticks=[])
```

- `add_subplot` : 한 화면에 여러 개의 이미지를 담기 위해 사용
 - a. 첫번째 파라미터: 행의 수를 의미, 이미지를 두 줄로 출력
 - b. 두번째 파라미터: 열의 수를 의미, 한 줄에 열 개의 이미지 출력
 - c. 세번째 파라미터: 인덱스를 의미, 행과 열을 기준으로 순차적으로 이미지 출력
 - d. 네번째 파라미터: 틱(tick)을 삭제하겠다는 의미
 - x 축, y 축 틱

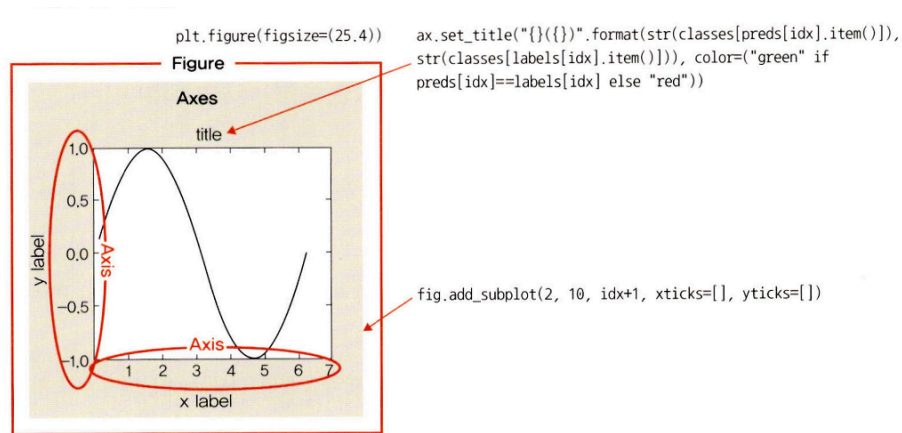


```
ax.set_title("{}({})".format(str(classes[preds[idx].item()]), str(classes[labels[idx].item()]))), color=("green" if preds[idx]==labels[idx] else "red"))
```

- `classes[preds[idx].item()]` : `preds[idx].item()` 값이 `classes`로 정의된 '0'과 '1' 중 어떤 값을 갖는지 판별하겠다는 의미
→ `classes[preds[idx].item()]` 값이 0이면 고양이, 1이면 개로 출력

```
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

- figure 안에서 서브플롯의 위치를 조정할 때 사용
- left, right, bottom, top으로 이미지 위치 조정
- hspace, wspace 사용하여 서브플롯 간의 간격 조정 가능.



2. 미세 조정 기법

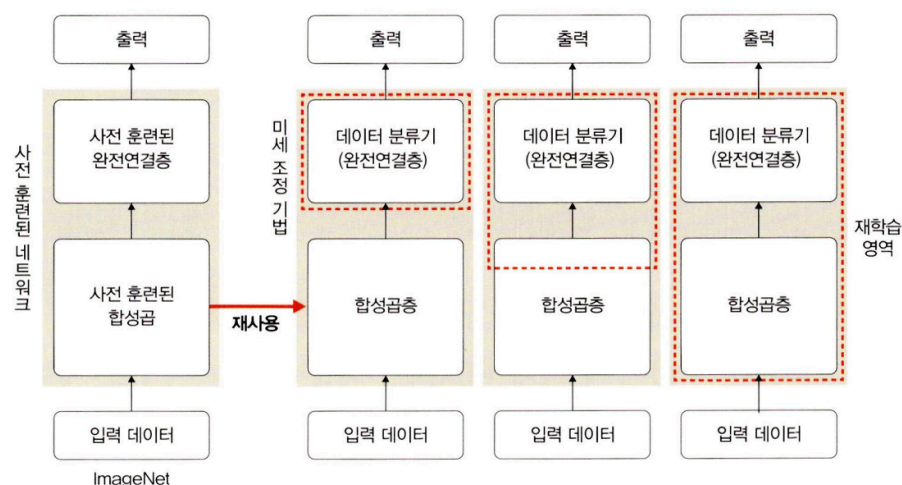
: 특성 추출 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트 하여 훈련시키는 방식

- 특성 추출은 목표 특성을 잘 추출했다는 전제하에 좋은 성능 낼 수 있음
→ 특성 추출 잘못될 경우, 미세 조정 기법으로 새로운 이미지 데이터 사용하여 네트워크 가중치 업데이트 → 특성 재추출

e.g. ImageNet 데이터 셋의 이미지 특징과 전자 상거래 물품의 이미지특징 다를 경우

⇒ 즉, 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습시키는 것

- 사전 훈련된 네트워크를 미세 조정 → 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터 조정하는 기법
- 미세 조정 과정에 많은 연산량 필요 → CPU 보다는 GPU 사용 권장
- 훈련 시키려는 데이터셋의 크기 / 사전 훈련된 모델에 따른 전략

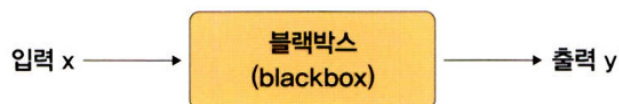


- 데이터셋 크기 ↑ / 사전 훈련된 모델 유사성 ↓ : 모델 전체 재학습
∴ 데이터셋 크기가 크기 때문에 재학습이 좋은 전략
- 데이터셋 크기 ↑ / 사전 훈련된 모델 유사성 ↑ : 합성곱층의 뒷부분 (완전연결층과 가까운 부분)과 데이터 분류기를 학습
∴ 데이터 셋이 유사하기 때문에 전체를 학습시키는 것보다는 강한 특징이 나타나는 합성곱층의 뒷부분과 데이터분류기만 새로 학습하더라도 최적의 성능 낼 수 있음
- 데이터셋 크기 ↓ / 사전 훈련된 모델 유사성 ↓ : 합성곱층의 일부분과 데이터 분류기를 학습
∴ 데이터가 적기 때문에 일부 계층에 미세 조정 기법을 적용 한다고 해도 효과가 없을 수 있 → 합성곱층 중 어디까지 새로 학습시켜야 할지 적당히 설정 필요
- 데이터셋 크기 ↓ / 사전 훈련된 모델 유사성 ↑ : 데이터 분류기만 학습
∴ 데이터가 적기 때문에 많은 계층에 미세조정기법을 적용하면 과적합 발생 가능 → 최종 데이터 분류기인 완전연결층에 대해서만 미세조정기법 적용

4 설명 가능한 CNN

: 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술

- CNN은 블랙박스 처럼 내부 동작 과정 설명 어려움 → CNN 결과 신뢰 어려움
⇒ CNN 처리 과정 시각화 필요성



- CNN을 구성하는 각 중간계층 ~ 최종분류 까지 입력된 이미지에서 특성이 어떻게 추출되고 학습하는지를 시각적으로 설명 → 결과에 대한 신뢰성
- CNN의 시각화방법 : 필터에 대한시각화 / 특성맵에 대한 시각화

1. 특성 맵 시각화

- 특성 맵: 입력 이미지 or 다른 특성 맵 등, 필터를 입력에 적용한 결과

⇒ 특정 입력 이미지에 대한 특성 맵 시각화 = 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수있도록 돕는 것

코드 5-30 필요한 라이브러리 호출

코드 5-31 설명 가능한 네트워크 생성

```
return F.log_softmax(x)
```

- `log_softmax()` : 신경망 말단의 결괏값들을 확률 개념으로 해석하기 위해 소프트 맥스 함수 결과에 log 값을 취한 연산
- 소프트맥스를 사용하지 않고 로그 소프트맥스 사용하는 이유: 소프트맥스는 기울기 소멸문제에 취약

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

$$\text{logsoftmax} = \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right)$$

$$= x_i - \log\left(\sum_{j=1} e^{x_j}\right)$$

코드 5-32 모델 객체화

코드 5-33 특성 맵을 확인하기 위한 클래스 정의

- 특성맵 : 합성곱층을 입력이미지와 필터를 연산하여 얻은 결과
⇒ 합성곱층에서 입력과 출력을 알 수 있다면 특성맵에 대한 값들을 확인할 수 있다는 의미

```
self.hook = model[layer_num].register_forward_hook(self.hook_fn)
```

- 파이토치는 매계층마다 print문을 사용하지 않더라도 hook 기능을 사용하여 각 계층의 활성화 함수 및 기울기 값 확인 가능
→ 따라서 `register_forward_hook` 의 목적은 순전파 중에 각 네트워크 모듈의 입력 및 출력을 가져오는 것
- 파이토치는 특정 값으로 정의되지 않은 중간 변수에 대해서는 기울기 값을 저장하지 않음
⇒ but, `register_hook()` 을 사용하면 기울기 값을 알 수 있음
→ hook 이용 시 중간 결괏값 확인 가능

코드 5-34 이미지 호출

```
img = cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR)
```

- `cv2.resize` : 이미지 크기를 변경할 때 사용
 - a. 첫번째 파라미터: 변경할 이미지 파일
 - b. 두번째 파라미터: 변경될 이미지 크기를(너비, 높이)로 지정

c. **interpolation** : 보간법

- 이미지 크기 변경 → 변형된 이미지의 픽셀을 추정해서 값을 할당해야 함
 - 이미지 비율 변경 → 존재하지 않는 영역에 새로운 픽셀값을 매핑하거나 존재하는 픽셀들을 압축해서 새로운 값을 할당해야 함
- ⇒ 이를 피하고자, 이미지 상에 존재하는 픽셀데이터(r, r)들에 대해 근사함수를 적용 → 새로운 픽셀값을 구하는 것 = 보간법
- 즉, 추정해야 하는 픽셀을 보간법을 이용하여 값을 할당

```
img = ToTensor()(img).unsqueeze(0)
```

- **unsqueeze()** : 1차원 데이터 생성하는 함수
- = 이미지 데이터를 텐서로 변환 → 변환된 데이터를 1차원으로 변경하겠다는 의미

코드 5-35 (0): Conv2d 특성맵 확인

코드 5-36 특성맵 확인

코드 5-37 20번째 계층에 대한 특성맵

코드 5-38 특성맵 확인

코드 5-39 40번째 계층에 대한 특성맵

코드 5-40 특성맵 확인

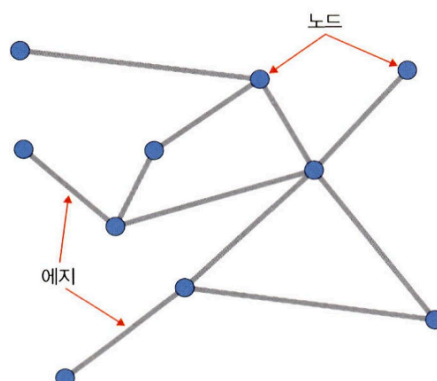
- 출력층에 가까울수록 원래 형태는 찾아볼 수 없고, 이미지 특징들만 전달되는 것을 확인

5 그래프 합성곱 네트워크

: 그래프 데이터를 위한 신경

1. 그래프란

- 그래프: 방향성이 있거나 없는 에지로 연결된 노드의 집합

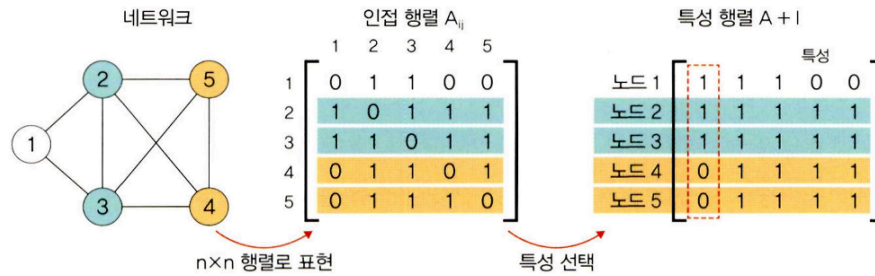


- 노드와 에지는 일반적으로 풀고자하는 문제에 대한 전문가 지식이나 직관등으로 구성

- 노드: 원소 (파란색 원)
- 에지: 결합 방법 (두 노드를 연결한 선)

2. 그래프 신경망

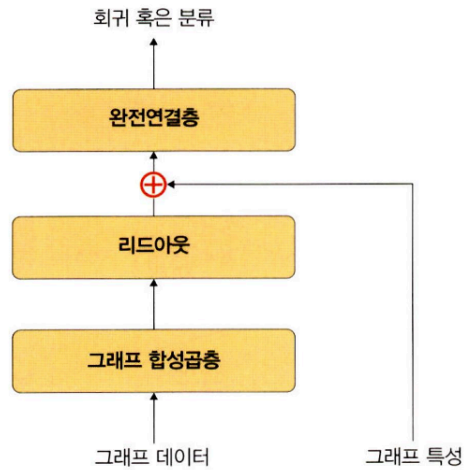
: 그래프 구조에서 사용하는 신경망



- 1단계: 인접 행렬 (adjacency matrix)
 - 노드 n 개를 $n \times n$ 행렬로 표현
 - 이에 생성된 인접 행렬 내의 값은 ' $A_{ij} = i$ 와 j 의 관련성 여부'를 만족하는 값으로 채워줌
 - ⇒ 컴퓨터가 이해하기 쉽게 그래프로 표현하는 과정
- 2단계: 특성 행렬 (feature matrix)
 - 인접 행렬만으로는 특성 파악 어려움 → 단위 행렬 적용
 - 각 입력 데이터에서 이용할 특성 선택
 - 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값 의미
 - ⇒ 특성 행렬을 거쳐 그래프 특성이 추출됨

3. 그래프 합성곱 네트워크

: 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘



- 리드아웃: 특성 행렬을 하나의 벡터로 변환하는 함수, 전체 노드의 특성 벡터에 대해 평균 구함 → 그래프 전체를 표현하는 하나의 벡터 생성
- 그래프 합성곱층: GCN에서 가장 중요한 부분
 ∴ 그래프 합성곱층을 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환 되어 딥러닝 알고리즘을 적용할 수 있음
- GCN 활용
 - SNS - 관계 네트워크
 - 학술 연구 - 인용 네트워크
 - 3D Mesh